

TradeLog - Viva Preparation Guide

TradeLog — Viva Preparation Guide & Local Run Instructions

Complete, step-by-step walkthrough of the project: architecture, every component, every API endpoint, the data flow, and how to run it on your own computer. Written to help you answer questions in a project viva/defense.

0. One-Minute Summary (say this first)

“TradeLog is a **trading journal web application**. A trader can log every trade with its entry, stop-loss, target, result and psychology; the app then shows dashboards, analytics, win rates, equity curves and backtest tools.

It is a **client-server application**: - **Frontend**: a single-page app built with **React 19 + TypeScript + Vite + Tailwind CSS + Recharts**. - **Backend**: a **REST API** built with **Node.js + Express 5 + Mongoose**, protected with **JWT authentication**. - **Database**: **MongoDB**, which by default runs **locally** (MongoDB Atlas optional).

The backend is the **single source of truth**. The frontend never stores trades on its own — every read/write goes through the API, and the API scopes all data to the logged-in user.”

1. How to Run the Project on Your Own Computer

1.1 Prerequisites

Software	Why	Check
Node.js 20+	Runs backend + frontend tooling	<code>node -v</code>
npm 10+	Package manager	<code>npm -v</code>
Git	Clone the repo	<code>git --version</code>
MongoDB 7/8	Local database	<code>mongod --version</code> (auto-installed by the script if missing)

1.2 Steps (Linux / macOS)

```
# 1. Clone the repository
git clone https://github.com/habingir19/tradelog.git
cd tradelog

# 2. (Optional but recommended) Install + start LOCAL MongoDB
./scripts/setup-local-db.sh

# 3. One-command start: local MongoDB + backend (:5000) + frontend (:5173)
./scripts/start-all.sh

# 4. Open the app
# http://localhost:5173

# 5. Stop everything (MongoDB stays running)
./scripts/stop-all.sh
```

`start-all.sh` is **idempotent** — run it any time; it reuses already-running services and auto-generates a random `JWT_SECRET` into `backend/.env` on first run.

1.3 Steps (Windows)

```
# 1. Clone and enter the repo
git clone http://github.com/habingir19/tradelog.git
cd tradelog

# 2. Install + start local MongoDB (PowerShell)
powershell -ExecutionPolicy Bypass -File scripts\setup-local-db.ps1
```

```
# 3. Open TWO terminals:
#   Terminal A -> backend
cd backend
npm install
copy .env.example .env      # then edit .env and set JWT_SECRET to any long string
npm run dev                 # http://localhost:5000

#   Terminal B -> frontend
cd frontend
npm install
npm run dev                 # http://localhost:5173
```

1.4 What each running piece is

Piece	Command	URL	Purpose
Local MongoDB	(service/script)	mongodb://127.0.0.1:27017/tradelog	stores users + trades
Backend API	npm run dev (backend)	http://localhost:5000/api	REST endpoints + JWT
Frontend	npm run dev (frontend)	http://localhost:5173	the user interface

The frontend’s Vite dev server **proxies** /api/* to http://localhost:5000 , so in development all API calls appear same-origin and no CORS configuration is needed.

1.5 Using MongoDB Atlas instead (optional, cloud)

Edit backend/.env , set:

```
MONGO_URI=mongodb+srv://<username>:<password>@<cluster>.mongodb.net/tradelog?retryWrites=true&w=majority&appName=Cluster0
```

Then whitelist your public IP in Atlas → **Network Access**. That’s it — no code changes.

2. System Architecture



Key architectural decisions 1. **Backend is source of truth** — trades and users live in MongoDB; the frontend just renders. 2. **JWT auth** — stateless: the server verifies the token on every protected call; no session table. 3. **Per-user data isolation** — every trade query filters by user: req.user._id . 4. **String trade _id s** (client-generated UUIDs) — makes CSV/JSON export→import round-trips stable. 5. **Optimistic UI** — the UI updates instantly, then syncs with the server and rolls back on failure.

3. Project Structure (explained folder by folder)

```
tradelog/
├── README.md      project overview + run instructions
├── docs/          documentation (overview PDF, this guide)
├── scripts/       dev helper
├── setup-local-db.sh  install + start local MongoDB (Linux/macOS)
├── setup-local-db.ps1 install + start local MongoDB (Windows)
```

```

├── start-all.sh      one-command start (mongo + backend + frontend)
├── stop-all.sh       stop backend + frontend
├── backend/           Express REST API
├── server.js          entry point (middleware, routes, db connect)
├── .env.example       template for environment variables
├── config/
│   ├── db.js          mongoose.connect(MONGO_URI)
│   └── index.js
├── models/
│   ├── User.js        user schema + bcrypt + toJSON (hides passwordHash)
│   └── Trade.js        trade schema (string id, UUID, user ref)
├── routes/
│   ├── index.js        mounts /health, /auth, /trades
│   ├── health.routes.js GET /api/health
│   ├── auth.routes.js  register, login, me, profile, password
│   └── tradeRoutes.js   CRUD + bulk + clear-all (all protected)
├── controllers/
│   ├── authController.js registration, login, profile, password logic
│   └── tradeController.js list/create/update/delete/bulk logic
├── middleware/
│   ├── auth.js         JWT verification (protect)
│   ├── validateTrade.js payload validation (enums, positive numbers)
│   └── errorHandler.js notFound + centralized error handler
├── utils/
│   └── asyncHandler.js wraps async controllers, forwards errors to next()
├── frontend/          React SPA
├── index.html          HTML entry point
├── vite.config.ts      dev server + /api proxy + allowedHosts
├── tailwind.config.js  Tailwind setup
├── src/
│   ├── main.tsx         React root + provider hierarchy
│   ├── App.tsx          router + protected routes + layout
│   ├── types/Trade.ts   Trade TypeScript type (mirrors backend schema)
│   └── lib/
│       ├── api.ts        the only file that talks to the backend
│       ├── authStore.ts   AuthContext + token get/set (local/sessionStorage)
│       ├── AuthProvider.tsx session restore, login/register/profile logic
│       ├── TradeContext.tsx TradeContext + useTrades() hook
│       ├── TradeProvider.tsx server-synced trade state (optimistic CRUD)
│       ├── csv.ts         CSV/JSON export + import (TradeLog + broker formats)
│       ├── analytics.ts   statistics helpers
│       ├── themeStore.ts / ThemeProvider.tsx dark/light theme
│       ├── settingsStore.ts / SettingsProvider.tsx backtest goal, backup counters
│       └── rulesStore.ts  trading-rules checklist (localStorage)
├── pages/
│   ├── Login.tsx         login / register form
│   ├── Dashboard.tsx     stats + charts + breakdowns
│   ├── TradeLog.tsx      table, filters, sort, bulk delete, export/import
│   ├── AddEditTrade.tsx  trade create/edit form
│   ├── AnalyticsPage.tsx deeper analytics
│   ├── RulesChecklist.tsx pre-trade rule checklist
│   ├── BacktestProgress.tsx backtest goal tracking
│   ├── ChartBacktest.tsx candlestick backtest
│   └── Profile.tsx        account, password, backtest goal
├── components/
│   ├── Navbar.tsx        sidebar navigation + logout
│   ├── StatCard.tsx       stat tile with trend
│   ├── EquityCurveChart.tsx equity curve (Recharts)
│   ├── CandleChart.tsx   candlestick chart
│   ├── TradeForm.tsx     the trade entry form
│   ├── TradeTable.tsx    sortable trade table
│   ├── BackupReminder.tsx reminds you to export trades
│   └── ThemeToggle.tsx   dark/light switch

```

4. Frontend — How It Works

4.1 Provider hierarchy (`main.tsx`)

Outer providers wrap inner ones; an inner provider can use an outer one via its hook.

```

<ThemeProvider> → dark/light theme (localStorage)
<AuthProvider> → user session + JWT
  <SettingsProvider> → backtest goal + backup counters (localStorage)
    <TradeProvider> → trades synced with the backend
      <App /> → <BrowserRouter> + routes

```

4.2 Routing (App.tsx)

- Public route: `/login` → `Login` page.
- Everything else is wrapped in `<ProtectedRoute>` (redirects to `/login` if no user) and then in `<Layout>` (sidebar Navbar + BackupReminder + page content).

Route	Page	Purpose
<code>/login</code>	Login	sign in / create account
<code>/</code>	Dashboard	performance overview, stat cards, charts, breakdowns
<code>/log</code>	TradeLog	table, filters, sort, bulk delete, CSV export/import
<code>/add</code>	AddEditTrade	create a trade
<code>/edit/:id</code>	AddEditTrade	edit a trade
<code>/analytics</code>	AnalyticsPage	deeper performance analytics
<code>/checklist</code>	RulesChecklist	trading rules checklist
<code>/backtest</code>	BacktestProgress	backtest goal tracking
<code>/backtest/chart</code>	ChartBacktest	candlestick backtest
<code>/profile</code>	Profile	account, change password, backtest goal

4.3 The API client (src/lib/api.ts) — the ONLY file that talks to the backend

- Reads the JWT (`getAuthToken()`) and adds header `Authorization: Bearer <token>`.
- Every response is parsed as JSON; on a **401** the stored token is cleared (auto-logout).
- On non-2xx it throws an `ApiError` with the backend message so the UI can show it.
- Base URL: `import.meta.env.VITE_API_URL` or `"/api"` (goes through the Vite proxy in dev).

Exposed methods:

```
api.auth.register({name,email,password}) → POST /api/auth/register
api.auth.login({email,password}) → POST /api/auth/login
api.auth.me() → GET /api/auth/me
api.auth.updateProfile(name) → PUT /api/auth/profile
api.auth.changePassword(current,next) → PUT /api/auth/password

api.trades.list() → GET /api/trades
api.trades.get(id) → GET /api/trades/:id
api.trades.create(trade) → POST /api/trades
api.trades.update(trade) → PUT /api/trades/:id
api.trades.remove(id) → DELETE /api/trades/:id
api.trades.clearAll() → DELETE /api/trades
api.trades.bulkCreate(trades[]) → POST /api/trades/bulk
```

4.4 Authentication state (authStore.ts + AuthProvider.tsx)

- Token key: `"tradelog.token"`.
- `setAuthToken(token, remember)` : if **remember** → `localStorage` (survives restart); else → `sessionStorage` (cleared when tab closes).
- On app start, if a token exists, `AuthProvider` calls `GET /api/auth/me` to restore the session. Invalid/expired token → token cleared → user null → redirected to `/login`.
- `AuthProvider` exposes `{ user, login, register, updateProfile, changePassword, logout }` via the `useAuth()` hook.

4.5 Trade state (TradeProvider.tsx + TradeContext.tsx)

- When a user logs in, `TradeProvider` fetches `GET /api/trades` and stores the list.
- All CRUD functions are **optimistic**:
 - `addTrade` → generates `crypto.randomUUID()`, adds to state immediately, then `POST /api/trades`; on success replaces with the server-confirmed record, on failure removes it + shows error.
 - `updateTrade` → updates in place, `PUT /api/trades/:id`; on failure reverts to previous.
 - `deleteTrade` / `deleteTrades` → removes optimistically, `DELETE`; keeps a copy in `lastDeleted` so the user can **undo** (re-import via `POST /api/trades/bulk`).
 - `clearAllTrades` → `DELETE /api/trades`.
 - `importTrades` → de-duplicates against existing ids, then `POST /api/trades/bulk`.

- Exposes { trades, loading, error, getTrade, addTrade, updateTrade, deleteTrade, deleteTrades, duplicateTrade, clearAllTrades, importTrades, canUndo, undoDelete } via the useTrades() hook.

4.6 Import / Export (csv.ts)

- **Export:** tradesToCsv() writes all fields (tags joined with |, quotes escaped) to a CSV.
- **Import** auto-detects the format from headers:
 - TradeLog export (has id, result, rr) → parseCsv.
 - Broker export (date/symbol/entry...) → parseBrokerCsv maps columns and computes rr/result.
 - JSON → array of trades or { trades: [...] }.

4.7 Vite proxy (vite.config.ts)

```
server: {
  proxy: {
    "/api": { target: "http://localhost:5000", changeOrigin: true }
  }
}
```

Any request to /api/* on the frontend is forwarded to the backend, so the browser only ever talks to localhost:5173 .

5. Backend — How It Works

5.1 Request lifecycle (order of middleware in server.js)

```
request
  → helmet() security header
  → cors({ origin }) CORS policy
  → express.json() parse JSON body
  → /api route
    → specific route handler
      → protect (JWT) [on protected routes]
      → validateTrade [on trade write routes]
      → controller business logic
      → Mongoose → MongoDB
    → response JSON
  → (if nothing matched) notFound 404
  → (if error thrown) errorHandler
```

5.2 Environment variables (backend/.env)

Variable	Purpose
NODE_ENV	development / production
PORT	API port (default 5000)
MONGO_URI	mongodb://127.0.0.1:27017/tradeLog (local) or Atlas string
CORS_ORIGIN	comma-separated allowed frontend origins
JWT_SECRET	secret to sign/verify JWTs (required — server refuses to start without it)
JWT_EXPIRES_IN	token lifetime (default 7d)

5.3 Full API reference (every endpoint)

All trade endpoints require Authorization: Bearer <JWT> .

Auth

Method	Path	Body	Success	Errors
POST	/api/auth/register	{name, email, password}	201 {success, token, data:user}	400 validation, 409 duplicate email
POST	/api/auth/login	{email, password}	200 {success, token, data:user}	400 , 401 wrong credentials
GET	/api/auth/me	— (JWT)	200 {success, data:user}	401

PUT	/api/auth/profile	{name}	200 {success, message, data:user}	400 , 401
PUT	/api/auth/password	{currentPassword, newPassword}	200 {success, message}	400 , 401 wrong current

Example register response:

```
{
  "success": true,
  "message": "Account created successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5IiwiaWF0IjoxNjY5MjY0MjY0",
  "data": { "id": "665f...", "name": "Nabin", "email": "me@mail.com", "createdAt": "..." }
}
```

Trades

Method	Path	Body / Query	Success	Errors
GET	/api/trades	query: search, direction, result, startDate, endDate, sort	200 {success, count, data:[...]}	400 bad filter
GET	/api/trades/:id	—	200 {success, data}	400 , 404
POST	/api/trades	trade object	201 {success, message, data}	400 validation
PUT	/api/trades/:id	trade object	200 {success, message, data}	400 , 404
DELETE	/api/trades/:id	—	200 {success, message}	400 , 404
DELETE	/api/trades	— (clears all for user)	200 {success, message, deleted}	—
POST	/api/trades/bulk	{ trades: [...] }	201 {success, message, count, data}	400

Example trade object (create):

```
{
  "date": "2026-08-04",
  "entryTime": "09:30",
  "pair": "EURUSD",
  "session": "London",
  "direction": "Buy",
  "entry": 1.10,
  "stopLoss": 1.09,
  "target": 1.12,
  "result": "Win",
  "rr": 2,
  "notes": "nice setup",
  "tags": ["sniper", "trend"],
  "isValidRuleTrade": true
}
```

5.4 JWT middleware (middleware/auth.js)

1. Reads `Authorization` header; expects `Bearer <token>`.
2. If missing → 401 "Not authorized, no token provided".
3. Verifies token with `jwt.verify(token, JWT_SECRET)` → gives `{ id }` (the user's ObjectId).
4. Loads the user from the DB (`User.findById(id)`); if the user no longer exists → 401.
5. Attaches the full user document to `req.user` and calls `next()`.

All trade routes and `/auth/me` , `/auth/profile` , `/auth/password` use `protect` .

5.5 Validation middleware (middleware/validateTrade.js)

For **create** (`POST`): requires `pair` , `direction` , `entry` , `stopLoss` , `target` . For **update** (`PUT`): validates only the fields that are present. Enums validated: `session` ∈ London/NewYork/Overlap/Other; `direction` ∈ Buy/Sell; `result` ∈ Win/Loss/BreakEven/Open. Numbers (`entry` , `stopLoss` , `target`) must be positive numbers. On any error → 400 with an `errors` array.

5.6 Error handler (middleware/errorHandler.js)

- Converts known Mongoose errors to clean HTTP responses:
 - `ValidationError` → 400 with field messages
 - `CastError` → 400
 - duplicate key (`code 11000`) → 409
- Any other error → 500 (stack trace only shown in development).
- `notFound` returns `404` for unmatched routes.

6. Database — Models and Data

6.1 User collection

Field	Type	Notes
name	String	required
email	String	required, unique , lowercase
passwordHash	String	bcrypt hash (10 rounds). Never returned by the API (<code>toJSON</code> deletes it).
createdAt / updatedAt	Date	automatic (<code>timestamps: true</code>)

`matchPassword(password)` uses `bcrypt.compare()` to check logins and password changes.

6.2 Trade collection

Field	Type	Notes
_id	String	default <code>crypto.randomUUID()</code> — keeps CSV round-trip ids
user	ObjectId ref → User	required, indexed — ownership + isolation
date	String <code>YYYY-MM-DD</code>	required
entryTime	String	optional
pair	String	required, uppercased
session	enum	London / NewYork / Overlap / Other
direction	enum	Buy / Sell (required)
entry , stopLoss , target	Number > 0	required, validated
result	enum	Win / Loss / BreakEven / Open
rr	Number ≥ 0	risk/reward
notes , emotion , reason , screenshot	String	optional
tags	[String]	default <code>[]</code>
mindset	<code>{ before , after }</code>	optional
isValidRuleTrade	Boolean	default false

The `toJSON` transform converts `_id` → `id` and hides `__v` , matching the frontend `Trade` type.

6.3 Why local vs cloud matters in the viva

- **Local**: `MONGO_URI=mongodb://127.0.0.1:27017/tradeLog` — data on your machine, free, private.
- **Atlas**: cloud MongoDB — data accessible from any device, needs internet + IP whitelist.
- The app is **database-agnostic**: only the `MONGO_URI` string changes.

7. End-to-End Workflows (step by step — narrate these in the viva)

7.1 Registration

1. User clicks “Create Account”, fills name/email/password.
2. `Login.tsx` calls `register(name, email, password)` from `useAuth()`.
3. `AuthProvider` calls `api.auth.register()` → `POST /api/auth/register`.
4. Backend validates (email format, password ≥ 6 chars); duplicate email → **409**.
5. Password is hashed with `bcrypt` (10 rounds); user document saved.
6. A JWT is signed: `jwt.sign({ id }, JWT_SECRET, { expiresIn: '7d' })`.
7. Response `201 { token, data }`; frontend stores the token and sets `user`.
8. User is redirected to the Dashboard; `TradeProvider` fetches (empty) trades.

7.2 Login + session restore

1. `POST /api/auth/login` with email/password.
2. Backend finds user by email, `bcrypt.compare()` the password.
3. Wrong credentials → **401**; correct → returns JWT.
4. Frontend stores the token (`localStorage` if “Remember me”, else `sessionStorage`).
5. On refresh, `AuthProvider` sees the token and calls `GET /api/auth/me` to restore the session.
6. Expired/invalid token → token cleared → redirect to `/login`.

7.3 Creating a trade

1. User opens `/add`, fills the form (`TradeForm.tsx`), submits.
2. `TradeProvider.addTrade()` creates the object with `crypto.randomUUID()`, updates the UI **optimistically**.
3. `POST /api/trades` → `protect` verifies JWT → `validateTrade` checks fields → `tradeController.createTrade` saves with `user: req.user._id`.
4. Server returns the confirmed record; `TradeProvider` swaps the optimistic record with it.
5. If the request fails → the optimistic record is removed and an error banner appears.

7.4 Editing / deleting / clearing

- Edit → `updateTrade` → optimistic replace → `PUT /api/trades/:id` → revert on failure.
- Delete one → `deleteTrade` → optimistic removal → `DELETE /api/trades/:id`; record saved for Undo.
- Delete many → `deleteTrades(ids)` → `Promise.allSettled` of deletes; reload on any failure.
- Clear all → `clearAllTrades` → `DELETE /api/trades`.
- **Undo** → `undoDelete()` re-inserts the remembered records via `POST /api/trades/bulk`.

7.5 Import / export (backup & migration)

1. Export: `tradesToCsv()` produces a CSV of the current trades → user downloads it.
2. Import: user drops a CSV/JSON; `parseImportFile` detects the format.
3. `importTrades` skips ids that already exist (de-dupe).
4. New rows are added optimistically and pushed with `POST /api/trades/bulk`.
5. `BackupReminder` counts trades since the last export and reminds the user to back up.

7.6 Analytics

1. Dashboard / Analytics read `trades` from `useTrades()`.
2. `useMemo` computations: win rate, average R, net R, profit factor, expectancy, max drawdown, streaks, best/worst trade, monthly and per-pair breakdowns.
3. Recharts renders: equity curve (`EquityCurveChart`), session bar chart, tables.
4. Everything derives from the **same in-memory trade list**, so it updates instantly when a trade changes.

7.7 Data isolation (important viva point)

Two users “Alice” and “Bob” both call `GET /api/trades`. The backend runs `Trade.find({ user: req.user._id })` — Alice only gets Alice’s trades because `req.user` comes from *her* token. Accessing another user’s trade id returns **404** (the query includes `user: req.user._id`, so it simply doesn’t match).

8. Security (always asked in viva)

Concern	How it’s handled
Password storage	<code>bcrypt</code> hash (10 rounds); raw password never stored/returned
Authentication	JWT signed with <code>JWT_SECRET</code> ; <code>expiresIn</code> default 7 days
Authorization	Every trade query scoped by <code>user</code> ; cross-user access → 404

Sensitive fields	User <code>toJSON</code> deletes <code>passwordHash</code> before any response
HTTP headers	<code>helmet</code> sets security headers; CORS restricted to <code>CORS_ORIGIN</code>
Input validation	<code>validateTrade</code> + Mongoose enums/validators; regex-escaped search
Secret handling	<code>backend/.env</code> is gitignored; <code>start-all.sh</code> auto-generates a random secret
Error leakage	Stack traces only in development; sanitized messages in production

9. Anticipated Viva Q&A (short answers)

- Q. Why did you choose the MERN-ish stack?** React for fast, component-based UI; Vite for fast dev/build; Express for a lightweight, unopinionated REST layer; Mongoose for schema validation on top of flexible MongoDB; JWT for stateless auth.
- Q. Why MongoDB instead of SQL?** Trades contain nested data (mindset, tags arrays) that maps naturally to documents; schemas are flexible and evolve quickly; the project has one primary read pattern (per-user trade list) that MongoDB indexes handle well.
- Q. How does authentication work?** Client sends `Authorization: Bearer <JWT>` on each request. The `protect` middleware verifies the signature, loads the user, and sets `req.user`. The JWT payload contains only `{ id }` and an expiry — no password data.
- Q. What is an optimistic update?** Update the UI state immediately before the server confirms, then reconcile with the server response; roll back if the request fails. It makes the app feel instant.
- Q. How is data kept separate between users?** Every controller filters by `req.user._id`, which is derived from the token. There is no way to query another user’s data without their token.
- Q. What happens if the backend is down?** The API client shows the error message; optimistic changes are rolled back; the session is kept (only a 401 clears the token). The app remains usable but can’t sync.
- Q. Why are trade ids strings, not ObjectIds?** The frontend generates UUIDs at create/import time, so an exported CSV can be re-imported (with the same ids) into a new database/account without id collisions.
- Q. How would you scale this?** Add indexes on `user+date`, paginate `GET /api/trades` (limit/offset), move long-running analytics to aggregation pipelines or a reporting service, deploy the backend behind a reverse proxy with the frontend as static assets, and add automated tests + CI.

10. Quick Troubleshooting (also useful in the viva)

Symptom	Fix
Backend won’t start: “JWT_SECRET is not defined”	create <code>backend/.env</code> (or run <code>start-all.sh</code> once)
“MongoDB connection failed”	start local MongoDB: <code>./scripts/setup-local-db.sh --ensure</code>
Port 5000/5173 already in use	the servers are already running (start-all is idempotent)
Frontend API calls fail	check the Vite proxy target matches the backend port
Empty trade list	you’re logged in as a different user (per-user isolation)
CSV import fails	the header row must include recognizable columns (see <code>csv.ts</code>)